

Traversal & Search

Cara bergerak di ruang kemungkinan: melebar, menyelam, mencoba pilihan, dan mencari batas.

Tujuan dokumen ini: bukan menghafal tanda-tanda soal, tapi memahami gerak berpikir pattern. Setiap pattern dijelaskan dengan contoh dunia nyata, contoh teknologi, konsep komputasi, alasan kenapa contoh itu sama dengan pattern, dan template Dart minimal.

Cara Membaca Dokumen Ini

Jangan baca seperti kamus keyword. Baca seperti latihan membangun intuisi. Pattern LeetCode itu mirip alat kerja: tiap alat punya cara bergerak, situasi natural, dan bentuk data yang cocok.

Untuk tiap pattern, pakai urutan baca ini: makna manusiawi -> contoh dunia nyata -> contoh teknologi -> konsep komputasi -> bentuk LeetCode -> template Dart -> pertanyaan cek pemahaman.

Pattern	Makna inti
BFS	menyebar melebar dulu
DFS	menyelam dalam dulu
Backtracking	coba pilihan, lanjut, undo kalau mentok
Binary Search	mencari batas antara bisa dan tidak bisa

BFS - menyebar melebar dulu

Makna manusiawinya

BFS adalah strategi mengecek semua kemungkinan yang jaraknya paling dekat dulu, baru bergerak ke lapisan berikutnya. Intuisinya: jangan terlalu cepat jatuh cinta pada satu cabang; sebarkan perhatian secara rata sampai ketemu kandidat terbaik atau terdekat.

Contoh nyata non-teknologi

Mencari bidang yang mau didalami

Lu belum tahu mau fokus ke Flutter, backend, AI automation, UI, atau content. Dengan BFS, lu coba permukaan semua dulu: bikin mini project kecil, baca intro, nonton 1-2 tutorial, lalu lihat mana yang terasa hidup. Baru setelah ketemu kandidat, lu masuk lebih dalam.

Mencari barang hilang di rumah

Lu mulai dari area terdekat: meja kerja, tas, saku, kamar. Setelah semua area dekat dicek, baru ke area lebih jauh. Ini level demi level, bukan langsung bongkar gudang selama 3 jam.

Contoh nyata di dunia teknologi

Notifikasi menyebar di social network

Saat satu post viral, sistem bisa melihat penyebaran dari orang pertama ke followers langsung, lalu followers dari followers, dan seterusnya. Setiap lapisan adalah jarak sosial.

Shortest path tanpa bobot

Di grid atau graph tanpa bobot, langkah pertama yang sampai tujuan adalah jalur terpendek, karena BFS bergerak berdasarkan jumlah langkah.

Konsep komputasinya

Secara komputasi, BFS memakai queue. Node yang masuk dulu diproses dulu. Karena queue menjaga urutan lapisan, semua node jarak 1 diproses sebelum jarak 2, semua jarak 2 sebelum jarak 3.

Kenapa contoh itu sama dengan pattern ini?

Contoh-contohnya sama dengan BFS karena semuanya punya konsep radius/lapisan: yang dekat dulu, lalu yang lebih jauh. BFS bukan tentang graph doang; graph hanya bentuk formal dari hubungan antar kemungkinan.

Bentuknya di LeetCode

- Cari jarak terpendek di graph/grid tanpa bobot
- Level order traversal di tree
- Rotting Oranges, Word Ladder, Shortest Path in Binary Matrix
- Multi-source spread: api, virus, rumor, notifikasi

Template Dart minimal

```
import 'dart:collection';

int bfs(int start, Map<int, List<int>> graph, int target) {
  final queue = Queue<int>();
  final visited = <int>{};
```

```

final distance = <int, int>{};

queue.add(start);
visited.add(start);
distance[start] = 0;

while (queue.isNotEmpty) {
  final node = queue.removeFirst();
  if (node == target) return distance[node]!;

  for (final next in graph[node] ?? const []) {
    if (visited.contains(next)) continue;
    visited.add(next);
    distance[next] = distance[node]! + 1;
    queue.add(next);
  }
}
return -1;
}

```

Kalau masih bingung, tanya begini

- Apakah gue butuh jawaban paling dekat/pendek?
- Apakah semua langkah punya biaya yang sama?
- Apakah prosesnya menyebar dari satu atau banyak titik awal?

Mini drill pemahaman

1. Bayangkan cari restoran: kapan lu pakai BFS, kapan DFS?
2. Kenapa queue lebih cocok daripada stack untuk jarak terpendek?
3. Apa yang terjadi kalau lu lupa visited?

DFS - menyelam dalam dulu

Makna manusiawinya

DFS adalah strategi memilih satu jalur lalu mengikutinya sedalam mungkin sampai mentok. Setelah mentok, baru mundur dan mencoba cabang lain. Intuisinya: fokus pada satu alur sampai habis.

Contoh nyata non-teknologi

Riset satu topik sampai akar

Lu memilih AI automation, lalu masuk ke Telegram bot, webhook, worker, queue, GitLab API, deployment, dan observability. Lu tidak pindah topik setiap 10 menit. Lu ikuti satu cabang sampai mengerti struktur dalamnya.

Mengecek folder sampai file terdalam

Saat membuka folder di komputer, lu masuk ke folder A, lalu subfolder, lalu subfolder lagi sampai file terakhir. Setelah selesai, baru balik ke folder lain.

Contoh nyata di dunia teknologi

Crawler halaman web

Crawler bisa masuk dari satu halaman ke link berikutnya, terus mengikuti link dalam sampai batas tertentu.

Tree recursion

Banyak operasi tree seperti max depth, invert tree, path sum, dan validate subtree natural sekali dengan DFS karena struktur tree memang nested.

Konsep komputasinya

Secara komputasi, DFS biasanya memakai recursion atau stack. Ia cocok untuk struktur yang bercabang dan dalam: tree, graph, folder, dependency, nested object.

Kenapa contoh itu sama dengan pattern ini?

Contoh-contohnya sama dengan DFS karena semuanya memprioritaskan kedalaman satu cabang sebelum menyelesaikan cabang yang lain. Kekuatan DFS adalah kesederhanaan untuk eksplorasi penuh dan struktur recursive.

Bentuknya di LeetCode

- Eksplorasi semua node/komponen
- Tree traversal preorder/inorder/postorder
- Path existence
- Connected component
- Nested structure

Template Dart minimal

```
void dfs(int node, Map<int, List<int>> graph, Set<int> visited) {  
  if (visited.contains(node)) return;  
  visited.add(node);  
  
  for (final next in graph[node] ?? const []) {  
    dfs(next, graph, visited);  
  }  
}
```

```
}  
}
```

Kalau masih bingung, tanya begini

- Apakah struktur masalah bercabang dan bisa diselami?
- Apakah gue cuma perlu tahu ada/tidak, bukan jarak terpendek?
- Apakah recursion membuat definisinya lebih natural?

Mini drill pemahaman

1. Jelaskan bedanya DFS di folder dan BFS di folder.
2. Kenapa DFS mudah menyebabkan infinite loop di graph tanpa visited?
3. Untuk mencari semua pulau di grid, kenapa DFS cukup?

Backtracking - coba pilihan, lanjut, undo kalau mentok

Makna manusiawinya

Backtracking adalah DFS khusus untuk ruang pilihan. Lu memilih sesuatu, lanjut, lalu kalau jalurnya salah atau sudah selesai, lu undo pilihan itu agar bisa mencoba pilihan lain. Intinya bukan hanya menyelam, tapi menyelam sambil membawa keputusan yang bisa dibatalkan.

Contoh nyata non-teknologi

Menyusun outfit

Lu pilih baju, celana, sepatu. Kalau kombinasi jelek, lu mundur: ganti sepatu dulu, kalau tetap jelek ganti celana. Pilihan sebelumnya harus bisa di-undo.

Mencari jadwal meeting

Lu coba slot Senin, lalu cocokkan dengan peserta. Kalau bentrok, undo slot itu dan coba Selasa. Kalau Selasa cocok, lanjut ke ruangan. Kalau ruangan penuh, undo ruangan atau slot.

Contoh nyata di dunia teknologi

Generate semua kombinasi fitur

Dalam product builder, lu bisa mencoba kombinasi pricing, limit user, feature bundle, dan trial. Setiap kombinasi adalah path.

Word Search

Di grid huruf, lu pilih huruf pertama, lanjut ke tetangga, kalau jalur tidak membentuk kata, undo posisi dan coba arah lain.

Konsep komputasinya

Secara komputasi, backtracking punya tiga hal: choice, constraint, goal. Choice adalah pilihan yang bisa diambil. Constraint adalah pilihan mana yang masih valid. Goal adalah kapan path dianggap selesai.

Kenapa contoh itu sama dengan pattern ini?

Contoh-contohnya sama karena semuanya punya pohon keputusan. Setiap langkah memperpanjang path. Kalau path tidak valid, kita mundur. Kalau valid dan lengkap, kita simpan jawabannya.

Bentuknya di LeetCode

- Subsets, permutations, combinations
- N-Queens
- Combination Sum
- Word Search
- Palindrome Partitioning

Template Dart minimal

```
void backtrack(List<int> nums, int start, List<int> path, List<List<int>> result) {  
    result.add(List<int>.from(path));  
  
    for (int i = start; i < nums.length; i++) {  
        path.add(nums[i]);          // choose  
        backtrack(nums, i + 1, path, result); // explore  
    }  
}
```

```
        path.removeLast();    // undo
    }
}
```

Kalau masih bingung, tanya begini

- Apa pilihan di setiap langkah?
- Apa yang membuat pilihan invalid?
- Kapan path selesai?
- Apa yang harus di-undo setelah recursive call?

Mini drill pemahaman

1. Untuk kombinasi menu makan siang, apa choice/constraint/goal-nya?
2. Kenapa path perlu dicopy saat disimpan?
3. Bedakan DFS biasa dan backtracking.

Binary Search - mencari batas antara bisa dan tidak bisa

Makna manusiawinya

Binary search bukan sekadar mencari angka di array sorted. Core-nya adalah mencari batas di ruang kemungkinan yang terurut: kiri gagal kanan bisa, atau kiri bisa kanan gagal. Ia memotong ruang pencarian setengah demi setengah.

Contoh nyata non-teknologi

Menentukan harga produk

Lu mau tahu harga maksimum yang masih masuk akal. Coba 500 ribu: terlalu mahal. Coba 250 ribu: masih oke. Coba 375 ribu: masih oke. Lu tidak mencoba semua harga satu per satu; lu mencari batas psikologis antara mau beli dan tidak.

Mencari level kemampuan

Lu mau tahu level soal tertinggi yang masih bisa lu kerjakan. Coba level 50, gagal. Coba 25, bisa. Coba 37, bisa. Coba 43, gagal. Lu mencari boundary.

Contoh nyata di dunia teknologi

Rate limit / capacity

Cari kapasitas minimum server agar request selesai dalam SLA. Kalau kapasitas X cukup, kapasitas lebih besar pasti cukup. Itu monotonic.

Koko Eating Bananas / shipping capacity

Cari speed/capacity minimum yang membuat constraint terpenuhi.

Konsep komputasinya

Secara komputasi, binary search butuh monotonic predicate: fungsi $\text{can}(x)$ yang berubah sekali dari false ke true atau true ke false. Kalau sifat ini tidak ada, binary search tidak valid.

Kenapa contoh itu sama dengan pattern ini?

Contoh-contohnya sama karena semuanya punya ruang kemungkinan yang bisa dipotong: harga, level, capacity, speed. Kita tidak peduli setiap nilai; kita peduli batas perubahan status.

Bentuknya di LeetCode

- Search in sorted array
- Find first/last position
- Find minimum in rotated sorted array
- Binary search on answer: capacity, speed, threshold

Template Dart minimal

```
int binarySearchBoundary(int low, int high, bool Function(int) can) {  
  while (low < high) {  
    final mid = low + ((high - low) ~/ 2);  
    if (can(mid)) {  
      high = mid;  
    } else {  
      low = mid + 1;  
    }  
  }  
}
```

```
    }  
    return low;  
}
```

Kalau masih bingung, tanya begini

- Apakah jawaban bisa diurutkan?
- Apakah ada `can(x)`?
- Kalau `x` bisa, apakah `x` yang lebih besar/kecil juga pasti bisa?

Mini drill pemahaman

1. Buat `can(x)` untuk harga produk.
2. Kenapa binary search gagal kalau predicate tidak monotonic?
3. Apa perbedaan cari elemen dan cari boundary?

Penutup

Mastery bukan berarti menghafal semua tanda soal. Mastery berarti saat melihat masalah, lu bisa memodelkan gerak masalahnya: apakah ia menyebar, menyelam, menjaga window, mencari boundary, mengingat key, menggabungkan kelompok, atau menyimpan subproblem.

Cara latihan: ambil satu pattern, pahami analoginya, kerjakan 3-5 soal mudah, tulis ulang konsepnya dengan bahasa sendiri, lalu campur dengan pattern lain supaya otak belajar mengenali bentuk masalah tanpa spoiler.